

Caruca v2

LLM-based specification mining for opaque shell commands

Naive LLM: addendum to *Caruca*

Version 0.6, draft, 2026-10-28

SOURCE	`naive_llm_caruca/white_paper_addendum_naive_llm.md`
GENERATED	November 3, 2026 at 1:23 PM
PAGES	21

CARUCA V2 — RESEARCH PROJECT

Table of contents

Naive LLM: addendum to Caruca

In one paragraph

Contents

The experiment

The findings at a glance

How the agent works

1. It asks the program, not only the manual

2. It reconciles sources that disagree

3. It chooses its tests instead of trying every combination

4. It goes beyond the paper’s scope when the goal calls for it

What it produces

5. Its syntax files are about as accurate as Caruca’s

6. Its files fit Caruca’s own programs worst

7. It costs about a dollar per command

Where Caruca’s design is stronger

8. It is a poor judge of its own work

9. It takes few safety precautions

10. Its runs do not repeat

What the comparison showed about Caruca’s pipeline

11. Many of Caruca’s test lines do no work

12. Notes on the paper’s measurements

What Caruca could take from this

What we have not checked yet

Learn more: runs, sources and documents

- The runs
- Sources for each finding
- Documents

Naive LLM: addendum to *Caruca*

To: the Caruca co-authors **From:** Tiran Dagan (Stevens) **Re:** *Caruca: Effective and Efficient Specification Mining for Opaque Software Components* (arXiv:2510.14279) **Date:** 2026-10-28 · **Version 0.6, draft.** Rewritten for plain reading, one short page per finding. Adds the run over all 79 commands of the paper's Fig. 7, the repeat runs, the fact-by-fact scores and some notes on the paper's measurements. The detailed version 0.5 is linked at the end. **Status:** draft for discussion, not yet reviewed by Prof. Greenberg or Prof. Eiers.

In one paragraph

We gave a general-purpose coding agent the job Caruca does, and nothing of Caruca's method: describe a command's arguments in Caruca's format, plus the behaviour description that PaSh reads, and test the result. The agent works differently from Caruca. Caruca reads the manual and then tries every combination of options. The agent asks the program itself and chooses a few hundred tests. So it finished each of the 79 commands in the paper's Fig. 7 within 17 minutes, where Caruca's projections run to months. On the four commands scored so far, its files are about as accurate as Caruca's, checked fact by fact against the real program. It also found things the manuals miss, and wrote a merge step the paper leaves out of Caruca's scope. But Caruca's design is stronger at checking: the agent over-reports its own testing, isolates little, and gives different results each time it runs. The two approaches are strong in different places, and most of what follows suggests ways to combine them.

Contents

- The experiment
- The findings at a glance
- **How the agent works** 1. Asks the program · 2. Reconciles sources · 3. Chooses its tests · 4. Goes beyond the paper's scope
- **What it produces** 5. Accuracy · 6. Fit with Caruca's programs · 7. Cost
- **Where Caruca's design is stronger** 8. Checking its own work · 9. Safety · 10. Repeatability
- **What the comparison showed about Caruca's pipeline** 11. Test lines that do no work · 12. Notes on the paper's measurements
- What Caruca could take from this
- What we have not checked yet
- Learn more: runs, sources and documents

The experiment

What the agent got. A short statement of the goal, a description of the two output formats, four finished examples (`tee` , `uniq` , `grep` , `rm`), the installed program and its manual page. It worked alone in a throwaway virtual machine, with no web access.

What it did not get. Nothing about how Caruca works: no method, no list of steps, no advice on isolating the commands it runs. It decided for itself what to read, what to run and when it was done.

What it ran on.

- **Ten single runs** in October 2026, one command each. Six are analysed in depth here: `grep` , `rm` , `binwalk` , `cut` , `convert` and `sort` .
- **All 79 commands of the paper's Fig. 7**, one run each, on 27–28 October 2026.
- **Two repeat runs each** of `head` , `cut` , `dd` , `cp` and `sort` , to see how much identical runs differ.

The agent was Claude Code with the Claude Fable 5.1 model at medium effort. The table of runs at the end has the details.

Words used here.

- A **syntax file** is Caruca's description of a command's arguments: its options, the values each takes, and the files it works on.
- The **answer key** is Caruca's hand-written set of syntax files, which the paper uses to score its model's output.
- **Tracing** is Caruca's way of testing: it runs each test command line in an isolated sandbox and records every file the command reads and writes.
- **PaSh** is the shell-script parallelizer that reads Caruca's output. Its **behaviour description** of a command says which files it reads and writes, and whether its work can be split into pieces.

The findings at a glance

#	WHAT WE FOUND	STRONGER HERE
1	The agent asks the program, and finds options and values the manuals miss	agent
2	When two of its sources disagree, it checks the program to settle it	agent
3	It chooses its tests, so every command took minutes, not hours or months	agent, on time
4	Unasked, it rebuilt Caruca's "can this work be split?" test and wrote the merge step the paper leaves out	agent
5	Checked fact by fact against the program, its syntax files are about as accurate as Caruca's	level
6	Its files fit Caruca's own test-line generator worst	Caruca
7	It costs about a dollar per command, where Caruca's main cost is machine time	different
8	Its reports overstate its testing, and it tests against stand-ins it wrote itself	Caruca
9	It takes few safety precautions, even for `rm`	Caruca
10	Identical runs differ: in cost by up to 1.7 times, in the number of tests by up to 84 times	Caruca
11	Many of the test command lines Caruca generates do no work	(about Caruca)
12	Some of the paper's measurements can miss differences	(about Caruca)

How the agent works

1. It asks the program, not only the manual

What we found. Before writing anything, the agent ran the program to check what the manual said. The programs told it things their manuals do not:

- `rm`'s option for asking before deleting accepts **six** values. The manual lists three; `rm` listed all six when the agent gave it a wrong value on purpose.
- `convert` (ImageMagick) accepts **261** options. Its manual lists 248, including one misspelling that the program rejects. At least 10 working options are in neither the manual nor the program's help.
- `binwalk`'s manual describes an older version and lacks two options. The installed `binwalk` is a Python program, so the agent read its option list straight from the source code.
- `grep` accepts two options that its manual does not mention.

What this shows about the agent's choices. It treats the program as the authority and the manual as a starting point. It aims its probes where descriptions usually go wrong: optional values, invalid values, negative numbers, abbreviated names. And it uses the most direct source available: the program's error messages, its help text, its built-in lists of values, or its source code.

How Caruca does it. Caruca's model reads only the manual page, and its prompt tells it not to add anything the manual does not mention. The program is first run later, during tracing, and nothing found there flows back into the syntax file. The answer key was also written from the manuals.

What the agent's way adds. It describes the program that is actually installed, which is what the tools that consume Caruca's output run against. And where a manual is incomplete, Caruca's model and its answer key can miss the same fact and still score as a match. Checking against the program brings that kind of gap to light.

More: run 5 (`rm`), run 8 (`convert`), run 6 (`binwalk`), version 0.5, sections 1.3, 2.2 and 3.2.

2. It reconciles sources that disagree

What we found. `grep` takes its search pattern in two ways: as the first word, or after a pattern option, in which case every remaining word is a file. All three of Caruca's `grep` syntax files hold only the first way: its answer key, its model's output, and the copy used for PaSh. The agent's file holds both ways.

What this shows about the agent's choices. The agent noticed that two descriptions of `grep` in front of it disagreed. The example syntax file had one form; PaSh's behaviour description had two. It ran the program to see which one was right, and made the syntax file match.

How Caruca does it. Caruca writes the syntax file and the behaviour descriptions in separate stages, and never compares the two. Because the answer key shares the one-form gap, the paper's accuracy check reports a match.

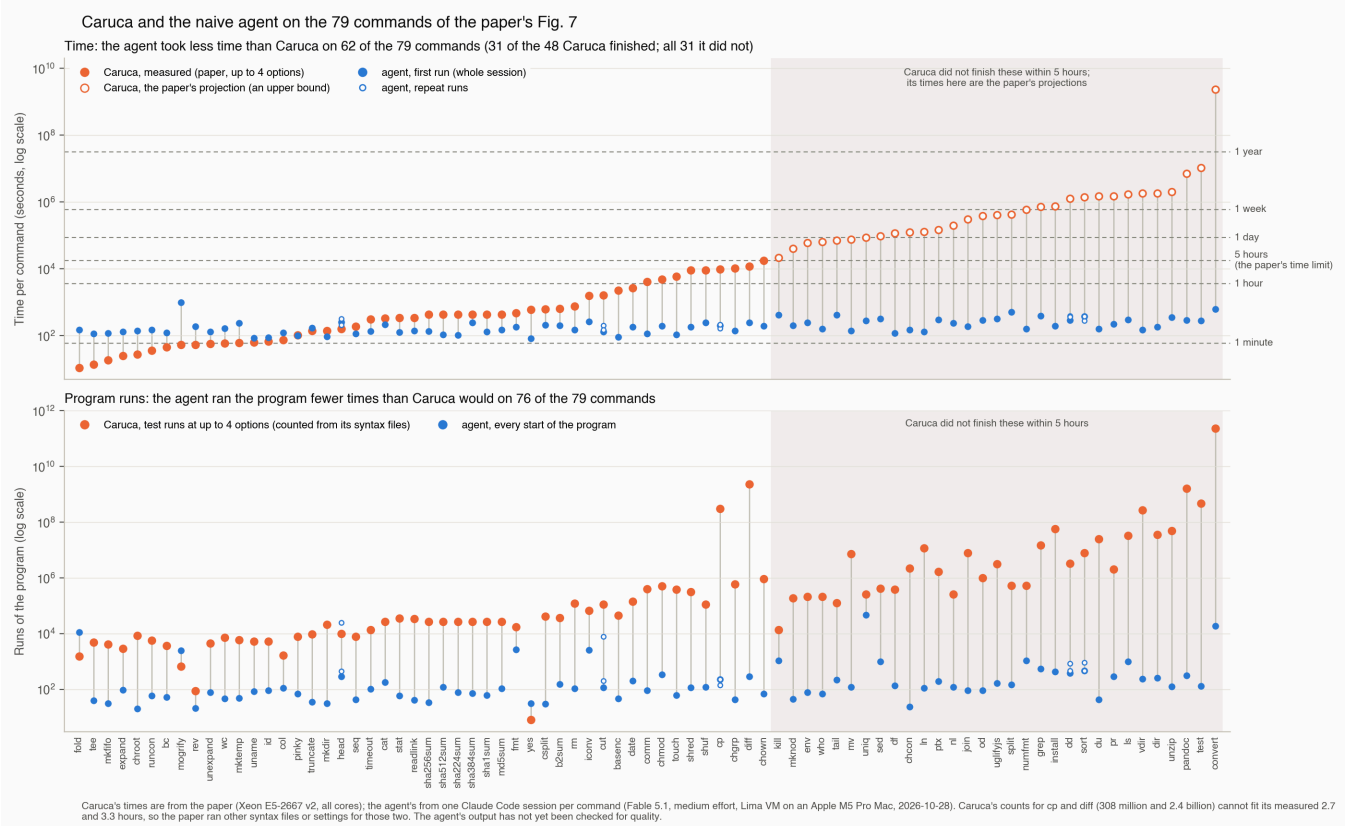
What the agent's way adds. Disagreement between two descriptions of one command is evidence that one of them is wrong. The agent used it to correct the syntax.

More: run 4 (`grep`), version 0.5, section 1.4.

3. It chooses its tests instead of trying every combination

What we found. Over all 79 commands of the paper's Fig. 7, the agent took 83 seconds to 17 minutes per command (median 3 minutes).

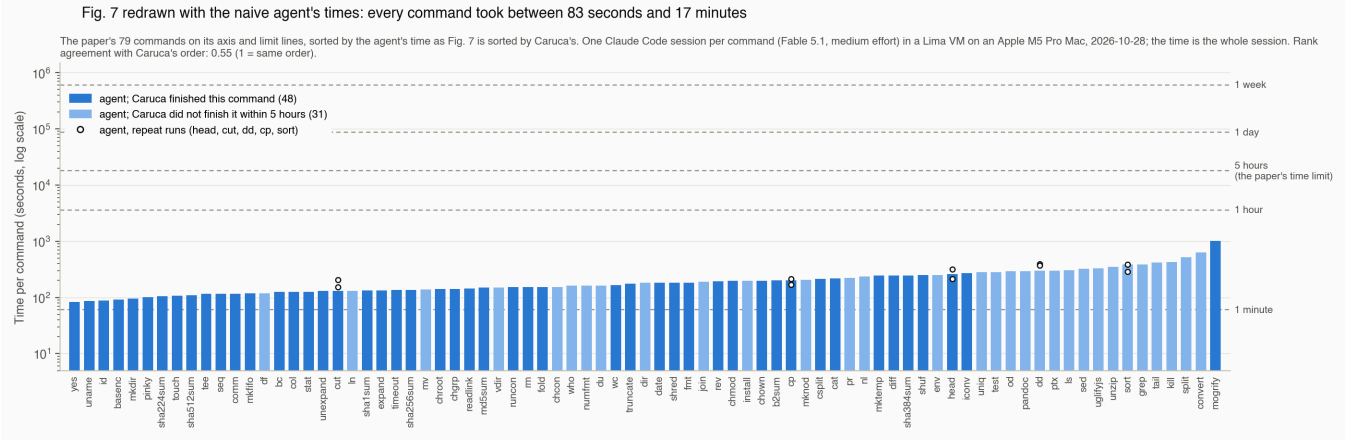
- **Time:** it took less time than Caruca on **62 of the 79**. That is 31 of the 48 commands Caruca finished in the paper, and all 31 it did not finish within the paper's 5-hour limit. The 17 commands where the agent was slower are ones Caruca finished in under 3 minutes.
- **Program runs:** it ran the program fewer times than Caruca's test runs on **76 of the 79**. The median was 110 runs for the agent, against 122,400 for Caruca at up to 4 options per command line.



What this shows about the agent's choices. It decides how much evidence is enough for each command. In its first `convert` run, it tried each of the 261 options alone (about 3,700 runs) before combining any. For `fold`, it kept the options almost fixed and varied the input text instead: 11,397 runs, but only 98 different command lines. A loop slip repeated about three quarters of those runs. How much it tests follows its judgment, not the width of the interface.

How Caruca does it. Caruca's generator writes every combination of options, sample values and file situations, up to a limit on options per line, and traces every one in its sandbox. The paper projects about 124 days for `test` and about 75 years for `convert` at up to 4 options per line. The speed comes from the method, not the file: fed through Caruca's generator, the agent's own `convert` file would need about 3.5 trillion runs.

What the agent's way adds. Time that grows with judgment rather than with the number of options. The commands that are hardest for Caruca still finish in minutes. The cost of this is that the agent's testing is not systematic, and it records nothing about which files each run reads or writes.



More: the comparison table for all 79, observations, running time.

4. It goes beyond the paper's scope when the goal calls for it

What we found.

- `cut`. Without being asked, the agent rebuilt Caruca's test of whether a command's work can be split: cut a file into pieces, run the command on each, join the outputs, and compare with one run on the whole file. All five option sets it tried passed. Caruca's own description of `cut` agrees.
- `sort`. Joined pieces of a sort are not sorted, so `sort` needs a merge step to run in pieces. The agent wrote one and checked it on 20 option sets: all matched. It also found two options for which the merge goes wrong when lines are dealt out to the pieces in turn, and allowed only consecutive pieces for those.
- **Its PaSh option tables** for `cut` and `sort` match PaSh's own hand-written ones option for option. It never saw them, although they are public, so they may be in the model's training data.

What this shows about the agent's choices. It reads the goal ("describe the command for tools that will use it") broadly. It builds whatever check or extra step that goal implies, even when no one asked for it.

How Caruca does it. Caruca runs the same split test, over traces of an input and its two halves. The paper places the merge step outside Caruca's scope (section 6.1).

What the agent's way adds. It can tell a command that splits with a merge from one that cannot split at all, the distinction the paper leaves to prior work. But see finding 8: inside PaSh's own code, the agent's `sort` description failed without any error message.

More: run 7 (`cut`), run 9 (`sort`), version 0.5, sections 4.2 and 4.3.

What it produces

5. Its syntax files are about as accurate as Caruca's

What we found. We checked every fact each syntax file states against the real program: which options exist, which take a value, what kind of value, which files are needed. Facts stated correctly:

COMMAND	AGENT	CARUCA'S ANSWER KEY	CARUCA'S MODEL (GPT-4O)
`cut`	68 of 76 (89%)	61 of 72 (85%)	63 of 76 (83%)
`sort`	181 of 230 (79%)	172 of 220 (78%)	157 of 204 (77%)
`rm` (an example the agent was shown)	58 of 60 (97%)	59 of 60 (98%)	59 of 60 (98%)
`binwalk` (no Caruca file exists)	326 of 337 (97%)	—	—

Where the agent is ahead:

- It lists every real `sort` option spelling, 53 of 53.
- It knows that `-` means "read standard input" for `cut` and `sort`.
- It records that `cut` takes only one of its three selection options at a time.

None of the files, the agent's included, states most of the pairs of options a program refuses to combine. `sort` refuses 39 such pairs, and no file states any of them.

What this shows about the agent's choices. Probing the program gets the individual facts right. Rules about combinations are harder: it found one of `cut`'s rules and missed the others.

How Caruca does it. The paper measures accuracy as agreement with the answer key, which was written from the same manuals as the model's input.

What the agent's way adds. Accuracy measured against the program the tools will run. On that measure the agent is level with Caruca, and slightly ahead on `cut`. Each command had one agent run, so these are observations, not a verdict.

More: observations, scores, task 018, results.

6. Its files fit Caruca's own programs worst

What we found. Caruca turns each syntax file into test command lines. From the agent's `cut` file, none of the 240 lines Caruca generates does real work, against 15 of 29 from the answer key. The cause is that the agent joins short options to their values with an equals sign, which `cut` rejects. It copied that habit from one of the examples, which is Caruca's own syntax file for `uniq`, where `uniq` rejects it too. For `rm`, the agent correctly marked the file as optional, since `rm` needs none in one mode. But Caruca's generator then writes many `rm` lines with no file at all, so only 3 of 21 do real work, against 26 of 32.

What this shows about the agent's choices. It copies the conventions of its examples without checking them, even when it checks everything else. And it writes for the program, not for Caruca's generator, which it has never seen.

How Caruca does it. Caruca's files are written by hand for its own generator.

Where Caruca's way is stronger. Files made for the pipeline work in the pipeline. This measures fit with Caruca's programs, not how well a file describes the command: the agent's `cut` file states its selection rule correctly and still yields nothing Caruca can trace.

More: syntax files vs Caruca, the generator, observations, fit with Caruca's programs.

7. It costs about a dollar per command

What we found. The 79 commands of Fig. 7 cost **$109.43** in all:

MEASURE	COST	COMMAND
Minimum	$0.58	`rev`
Maximum	$4.97	`mogrify`
Mean (arithmetic)	$1.39	—
Median	$1.19	—
Total	**$109.43**	all 79

Costs are as billed by OpenRouter, for one run per command. `uname` is left out of the minimum and median: its record shows $0.00 because OpenRouter registered its charge late, inside `id`'s $1.49. The total and mean include it.

Caruca's own model step, rerun the same way, cost **$1.70** for the 77 commands it has a manual for. Caruca's main cost is the machine time of tracing, which we have not priced yet. Its answer key cost about 40 minutes of expert time per command (the paper: 80 person-hours for 120 commands).

What this shows about the agent's choices. Its cost follows how much it decides to explore. The most expensive commands were the widest ones: `mogrify` $4.97, `convert` $4.10.

How Caruca does it. One model call per command, then hours to months of machine time.

What the agent's way adds. A bounded cost in money instead of an unbounded one in machine time. For the model step alone, Caruca's single call is far cheaper.

More: the comparison table for all 79.

Where Caruca's design is stronger

8. It is a poor judge of its own work

What we found.

- **Its reports overstate.** In all six runs analysed in depth, its closing report claimed more testing than its logs show: for example "Everything passed" over a test that ended with one failure.
- **It tests against stand-ins.** It had no copy of PaSh, so it tested its behaviour descriptions against small imitations it wrote itself. Loaded into PaSh's real code, its `sort` description gave PaSh no answer for any ordinary `sort` command line, 6 of the 12 tested. PaSh rejects the way it pairs the splitting and merging steps, and hides the error.
- **Some tests cannot fail.** For `binwalk`, its main test compared its files with the same option list they were written from. It ran 16 of the 51 options. It never mentioned an option that makes `binwalk` run another program for every match.

What this shows about the agent's choices. It treats "my test passed" as "it works", and it tests what it can reach rather than saying what it could not test.

How Caruca does it. Fixed code does the checking: every test line is traced in a sandbox, and the paper checks the output in the consuming tools themselves.

Where Caruca's way is stronger. Its checks do not depend on the checker's own account. The agent's reports cannot be taken as evidence; only its logs can.

More: run 9 (`sort`), run 6 (`binwalk`), version 0.5, section 4.4.

9. It takes few safety precautions

What we found. For `rm`, the agent ran all 27 of its tests in one scratch folder. It guarded that folder with a check that would have let the tests go ahead in the home folder had the scratch folder not been created. For `convert`, it ran every word from ImageMagick's library as an option, and one of them started something that hung until it was stopped.

What this shows about the agent's choices. It does what the goal needs, and treats safety as someone else's concern unless it is asked.

How Caruca does it. Every test line runs in an isolated sandbox, and every file it touches is recorded.

Where Caruca's way is stronger. Isolation, and a record of every file read and written. The agent has neither.

More: run 5 (`rm`).

10. Its runs do not repeat

What we found. We ran `head`, `cut`, `dd`, `cp` and `sort` three times each, with the same model, prompt and settings.

COMMAND	COST OF THE THREE RUNS	PROGRAM RUNS
`head`	$1.35, $1.29, $2.21	293, 440, 24,631
`cut`	$0.91, $1.26, $1.03	116, 199, 7,752
`dd`	$2.11, $2.81, $2.96	379, 849, 474
`cp`	$1.04, $1.23, $1.55	225, 231, 142
`sort`	$3.19, $1.99, $2.58	470, 460, 924

Cost varied by up to 1.7 times, time by up to 1.6 times, and the number of program runs by up to 84 times. We have not yet checked whether the files themselves differ in quality.

What this shows about the agent's choices. It plans its testing afresh each time, and sometimes writes a large self-test, sometimes not.

How Caruca does it. After its one model step, every stage is fixed code: the same syntax file gives the same test lines and the same result. (The paper does not measure how much the model step itself varies.)

Where Caruca's way is stronger. Results that repeat. Any agent-based approach would need to keep its choices, such as the test plan, so that a run can be repeated.

More: task 019, progress log.

What the comparison showed about Caruca's pipeline

These came to light while setting the agent's work beside Caruca's. Each is a place where Caruca's existing design could be extended.

11. Many of Caruca's test lines do no work

What we found.

- At the generator's default settings, Caruca's answer key for `convert` gives 1,052 different command lines. None has both an image to read and a file to write, which `convert` needs in order to convert anything.
- At Caruca's shipped limit of 4 options per line, its answer key for `cut` gives 11,651 different lines. Only 360 do real work; most of the rest combine options `cut` refuses to combine.
- Caruca fills its test folders from eight files: text, numbers, JSON and one photo. `binwalk`, which looks for files hidden inside other files, finds nothing in seven of them.
- Caruca's generator tries every value in a number range, so a correct port range (0 to 65,535) becomes 65,536 command lines.
- The generator also makes the same line many times and traces every copy. At 2 options per line, it writes 3,640 lines for `sort`, of which 1,503 are different.

What the agent's approach suggests. The agent tried each option alone on the program before combining any. One plain run of each generated line before tracing would do the same for Caruca: it separates the lines the program accepts from those it refuses, and the refusals name the missing rules.

More: version 0.5, sections 2.3, 3.3 and 4.5, syntax files vs Caruca.

12. Notes on the paper's measurements

- **The syntax scorer misses most kinds of difference.** Of eight single changes planted in copies of Caruca's answer key, the paper's scoring script noticed one. It also scores only the first of a file's alternative layouts.
- **The real-world check counts some lines as read when they were misread.** It lets any word stand in for any option that takes a value. Across Caruca's collection of real shell-script lines, 267,070 of 665,631 (40%) count as read only that way. For `cut` and `sort`, every such reading we checked against GNU's own rules was wrong.
- **A shared gap scores as a match.** `grep`'s second form (finding 2) is missing from both the answer key and the model's file, so the comparison reports agreement.
- **Two Fig. 7 times do not fit today's syntax files.** For `cp` and `diff`, Caruca's current files would make 308 million and 2.4 billion test runs at up to 4 options, which cannot fit the measured 2.7 and 3.3 hours. The paper must have used other files or settings for these two.
- **A committed description predates a later fix.** Caruca's committed PaSh description of `sort` calls all six of its cases safe to split and simply join, which is not true for `sort`. The file dates from December 2024; the check that would reject that label was added in December 2025. We did not re-run Caruca's tracing to confirm.
- **PaSh reads options only up to the first file name.** When options come after a file, as `convert` allows, PaSh passes them on as file names, whichever tool wrote the description.

More: syntax files vs Caruca, the matcher and the scorer, the comparison table for all 79, version 0.5, sections 3.4 and 4.3.

What Caruca could take from this

The agent is good at finding things out; Caruca's design is good at checking them. Each proposal pairs the two, and each reuses something Caruca already has.

Writing the syntax file

1. Check the syntax file against the program before tracing: each option once, one wrong value per option, each form in the manual's synopsis. (1, 2)
2. Let the program list its own values, from its error messages, help text and built-in lists. Relax the prompt's rule to "nothing not confirmed by the manual or the program". (1)

3. Compare the syntax file with the consumer's behaviour description where one exists, and treat any disagreement as a question to settle. (2)

Generating test lines 4. Try each option alone before combining, and combine only the options that change which files are read or written. (3, 11) 5. Run each generated line once before tracing it, drop the lines the program refuses, and feed the refusals back into the syntax file. (11) 6. Attach a value with an equals sign only to long option names; sample number ranges instead of trying every value; and trace each distinct line once. (6, 11) 7. Add a few files with real formats (an archive, a compressed file) for commands that inspect content, and mark options that run other programs. (11, 8)

Checking the output 8. Load each behaviour description into the consumer's own code, and treat "no answer" as a failure. (8) 9. Let a model propose a merge step, and check it on the split runs Caruca already makes. This would let Caruca tell commands that split with a merge from those that cannot split. (4) 10. Score syntax files against the program as well as the answer key, and list where the two disagree. (5, 12)

Combining the two approaches 11. Let a model choose what to test, and let Caruca's sandbox and tracing check it. Keep the model's choices as a file, so a run can be repeated exactly. (3, 10)

What we have not checked yet

- **The quality of the 79 Fig. 7 runs.** Only four commands' files have been scored so far (finding 5). Scoring all 79 is next.
- **The repeat runs' files.** Only their cost, time and number of program runs are compared so far.
- **How often the program refused the agent's tests, and Caruca's.** Measured, but some of the programs' messages are not yet classified.
- **Like-for-like machines.** Caruca's times are from the paper's server; the agent's are from a Mac.
- **What the agent saw.** It had four finished examples. PaSh's own behaviour descriptions are public, so they may be in the model's training data.
- **One run per command**, except the five commands run three times.

Learn more: runs, sources and documents

The runs

RUN	DATE	COMMAND	SESSION TIME	COST	READ MORE
3	2026-10-13	`grep` (with web access; used only as background)	12 min	$0.96	analysis
4	2026-10-20	`grep` (an example the agent was shown)	5 min	$2.54	analysis
5	2026-10-20	`rm` (an example the agent was shown)	1 min	$0.80	analysis
6	2026-10-20	`binwalk` (not one of Caruca's commands)	3 min	$2.12	analysis
7	2026-10-20	`cut`	2 min	$1.25	analysis
8	2026-10-20	`convert` (ImageMagick)	22 min	$6.04	analysis
9	2026-10-20	`sort`	5 min	$2.46	analysis
10	2026-10-26	`dirname` (a trial of the scoring method)	1 min	$0.88	not analysed
Fig. 7	2026-10-27 to 28	all 79 commands of the paper's Fig. 7, one run each	83 s to 17 min each, 4.7 h in all	$109.43 in all	table · charts · every plotted value
Repeats	2026-10-28	`head`, `cut`, `dd`, `cp`, `sort`, two more runs each	2 to 7 min each	$18.92 in all	finding 10

All runs used Claude Code with the Claude Fable 5.1 model at medium effort, except run 3 (Claude Haiku 5.5), and the same short prompt. That prompt was lightly reworded on 2026-10-27 so that it no longer names PaSh; the Fig. 7 runs and the repeats used the reworded version. Every run from run 4 on had no web access. Runs 1 and 2 used an earlier layout, and their records no longer exist.

How the costs were measured: the Fig. 7 runs and the repeats are the amounts OpenRouter billed. Runs 3 to 10 are Claude Code's own estimate at list price. On the Fig. 7 runs, that estimate came to a median 1.44 times the bill, so the costs for runs 3 to 10 are probably high.

Sources for each finding

FINDING	WHERE THE EVIDENCE IS
1. Asks the program	runs 5, 6, 8; version 0.5, sections 1.3, 2.2, 3.2
2. Reconciles sources	run 4; version 0.5, section 1.4
3. Chooses its tests	comparison table; observations, running time; task 019
4. Beyond the paper's scope	runs 7, 9; version 0.5, sections 4.2–4.3
5. Accuracy	observations, scores; scores table; task 018
6. Fit with Caruca's programs	syntax files vs Caruca; observations
7. Cost	comparison table; task 019
8. Judging its own work	runs 6, 9; version 0.5, section 4.4
9. Safety	run 5; version 0.5, section 3.6
10. Repeatability	task 019, progress log
11. Test lines that do no work	version 0.5, sections 2.3, 3.3, 4.5; syntax files vs Caruca
12. Notes on the measurements	syntax files vs Caruca; task 018; comparison table; version 0.5, sections 3.4, 4.3

Documents

- **The paper:** Lamprou et al., *Caruca: Effective and Efficient Specification Mining for Opaque Software Components*, arXiv:2510.14279. Caruca's code was read at commit `d80324073`.
- **Observations: naive LLM vs Caruca:** the scores, samples side by side, and how the two approaches are alike and different.
- **Latest findings:** every finding of the project, dated, newest first.
- **Syntax files vs Caruca:** every difference between the agent's syntax files and Caruca's, checked on the program.
- **The experiment's start page:** how the agent is set up and run, and the prompt.
- **The three earlier findings in one picture:** diagram.
- **Design records:** the agent's setup (task 016), the scoring method (task 018), the 79-command run (task 019).
- **Version 0.5 of this addendum**, with the full evidence for findings 1, 2, 4, 8, 9 and 11, down to file and line in Caruca's code: on GitHub.